Live verification against the demo environment

Samuel — 15 August 2026 Target: `http://149.56.128.57:3200`

I ran the first round of this assessment without a browser, working only from GET requests and the documentation. This pass closed that gap. It also proved two of my earlier conclusions wrong, and I have rewritten them rather than quietly dropping them.

---

1. The demo has no real backend

The thing sitting behind the Angular app is a `json-server` fixture, not NexaPay's `transfer-svc`. Four things say so:

- `GET /db` returns the whole database in one response: `users`, `transactions`, `contacts`, `transfers`. That endpoint is json-server's signature.
- The app fetches `/transactions?_sort=date&_order=desc`. `_sort` and `_order` are json-server query conventions.
- `/transactions` and `/api/v1/transactions` return identical payloads, so the `/api/v1` prefix is a rewrite rule rather than a service.
- A POST of `{"hello":"world"}` came back `201` with an auto-incremented integer id. That is json-server's default write behaviour.

None of the user records has a password field, so sign-in is happening entirely in the browser.

This matters more than any single defect I found. A bare json-server has no authentication, no authorisation, no schema validation and no idempotency, because it was never meant to. So every "the server does not enforce this" finding from my first round is telling us about the fixture, not about NexaPay.

Writing up "your API accepts a one million dollar transfer" as a backend defect would have been wrong. The accurate version is less comfortable and more useful:

> This environment cannot validate a single server-side business rule, so no release decision can be built on it.

There is one exception to the json-server picture. Every POST to `/transfers` also creates a matching record in `/transactions`, which a plain fixture would not do. So there is a small amount of custom server code. It still validates nothing.

---

2. What POST /transfers actually does

Eight requests, all of them answered `201 Created`.

Request	Result
Valid, with an `Idempotency-Key`	`201`, id 1
Identical replay, same key	`201`, id 2
`amount: 100000000` (one million dollars)	`201`, id 3
`amount: -2500`	`201`, id 4
No `pin` field at all	`201`, id 5
`scheduledDate: "2020-01-01"`	`201`, id 6
61-character note	`201`, id 7
`{"hello":"world"}`	`201`, id 8

The last one is the one to look at twice. A body with no recipient, no amount and no PIN was accepted and stored. That is not a gap in validation, it is the absence of a schema.

All of this is json-server behaving normally. Three other things are not, and those concern the Angular client, which is the actual deliverable:

The PIN travels and is stored in clear text. The record that comes back contains `"pin": "123456"`. R08 says the PIN should be SHA-256 hashed in the browser so the raw value never moves. It moves, and it is now sitting in the database.

R08 cannot be implemented on this deployment anyway. The app is served over plain HTTP, so `window.isSecureContext` is false and `window.crypto.subtle` is undefined. The Web Crypto API is simply not available. You would have to ship a JavaScript SHA-256 implementation in the bundle to satisfy the rule as written. Plain HTTP also breaks the "encryption in transit" line in Annexe D.

The response does not match Annexe E. The contract says `id` is a UUID string; it comes back as the integer 1. The contract lists a `status` field; it is not there at all. Anyone building a client from that YAML breaks immediately.

---

3. The transfer form does not submit

Recipient filled in, amount 12.34, note filled, type Instant selected, a valid six-digit PIN, no validation errors on screen, submit button enabled. Clicking Send Transfer produces nothing:

- no network request, checked by wrapping `fetch` and `XMLHttpRequest`
- no toast
- no navigation
- no error message
- nothing in the console

I tried it with a synthetic click and again with a native `click()`. Same result. The handler does nothing.

This is a real front-end defect. The Angular bundle is what is being assessed, and its most important control is inert, so the transfer journey cannot be completed through the UI at all. My API-level tests all bypass the form, which is exactly why the brief asks for an end-to-end journey and exactly why I could not have found this in round one.

Smaller, related: no transfer type is selected when the form loads, the form will not submit without one, and no "please choose a type" message ever appears. Even after the submit handler is fixed, anyone who does not think to click a radio button is stuck with a form that looks complete and does nothing.

---

4. The balance figures — correcting what I said first time

I originally reported that the balance was hard-coded. **That was wrong, and the way I got it wrong is worth recording.**

My reasoning was that cycling the status filter changed the row count correctly (15, 10, 3, 2) while the balance stayed at $50,563.80. I read a figure that does not move as a figure that is not computed.

The filter test could never have shown what I claimed. Filters change which rows are displayed. An account balance is computed over the whole dataset, so of course it does not move when you filter the view. I designed an experiment that could not distinguish the two explanations and then drew a conclusion from it.

What settled it was an accident. My eight test transfers, including the one million dollar one, pushed the dashboard to **-$100,010,822.00**. After I deleted the test data it went back to **$48,291.00**. So the balance does respond to the data. It is derived.

The defect survives the correction, in a different shape:

Where	Figure
Dashboard, “Total Balance”	$48,291.00
Transactions page, “Balance”	$50,563.80
Sum of all 15 ledger records	-$389.66
Sum of completed records only	$1,883.14

Two screens in the same app show two different balances, $2,272.80 apart, and neither matches any obvious aggregation of the ledger. The formula is undocumented and I could not reconstruct it from the API.

Income and Expenses are a separate case. Those stayed at $12,750.00 and $8,420.00 the entire time, including while 23 extra transactions were sitting in the database. Those two really do look static. The Savings Rate of 33.9% is just (12750 − 8420) / 12750, so that block computes correctly from figures that are themselves disconnected from the ledger.

One more thing came out of this. After I deleted the test data, the dashboard kept showing -$100,010,822.00 until a hard reload. It does not refetch on navigation. On a financial dashboard, showing a stale balance after the underlying data has changed is its own problem.

---

5. Annexe H’s TC-04 — root cause

Round one called this a broken test with an unexplained timeout. The DOM explains it.

The existing suite waits for `[data-testid=row]`. The application renders `[data-testid=tx-row]`. The selector cannot ever match, so the wait always times out. Because the runner aborted there, everything scheduled after it in that file never ran, which is why the report totals 11 tests while listing TC-01 to TC-12 with no TC-10 anywhere.

The real hooks, read off the running app:

```
sidebar, nav-dashboard, nav-transactions, nav-transfer, nav-upload,
sidebar-user-chip, sidebar-logout, topbar, topbar-title, topbar-search,
notification-bell, notification-badge, send-money-btn, lang-toggle,
filter-category, filter-status, search-input, tx-balance,
sort-date, sort-amount, tx-row, tx-details-btn, tx-delete-btn,
page-prev, page-label, page-next,
transfer-recipient, transfer-amount, transfer-note, transfer-type,
transfer-type-instant, transfer-type-standard, transfer-type-scheduled,
transfer-pin, transfer-submit
```

`data-testid=resend`, which TC-07 asserts on and which passes, is not on the transactions list. It may be on the detail view. A passing test whose selector I cannot find is worth a second look before anyone trusts it.

The page objects in `src/pages/` now use these hooks, and the app’s hash routing (`#/login`, `#/transactions`, `#/transfer`). The plain paths I assumed in round one do not resolve.

---

6. What works

Listing only failures would give a false picture. These all behaved correctly:

- Sign-in as Member, session held, correct name and role shown
- The route guard on `#/admin` sends a Member back to the dashboard
- No Admin nav entry for Member
- $10,000.00 produces “Amount cannot exceed $9,999.99”

- A five-digit PIN produces “PIN must be exactly 6 digits”
- The form refuses to submit while invalid, with no request sent
- Status filters partition correctly, 15 / 10 / 3 / 2, matching the API exactly
- Pagination at five rows a page, “Page 1 of 3”
- Rows render the right amounts, dates, categories and statuses

The route guard is worth a note. My test design argued that hiding a nav link is decoration and that only the guard and the endpoint are real controls. The guard turns out to be properly built, better than I predicted. The endpoint behind it is wide open, but that is the fixture, not the Angular app.

---

7. Revised defect list

ID	Severity	What	Whose
DEF-013	Critical	Transfer form submit does nothing	Front end
DEF-010	Critical	Two balances, $2,272.80 apart, neither matching the ledger; Income and Expenses static; dashboard stale until reload	Front end
DEF-014	Critical	PIN sent and stored in clear	Front end
DEF-015	Major	Plain HTTP, so `crypto.subtle` is unavailable and R08 cannot be implemented	Infrastructure
DEF-004	Major	`id` is an integer not a UUID, `status` missing — contract breach	Mixed
DEF-017	Minor	No transfer type preselected and no message saying one is needed	Front end
DEF-001	—	Unauthenticated reads	Environment
DEF-003	—	No tenant scoping	Environment
DEF-005	—	Idempotency-Key ignored	Environment
DEF-007	—	No amount ceiling server-side	Environment
DEF-009	—	Bad enum filter returns `200 []`	Environment
DEF-011	—	No `/health`, `/ready`, `/metrics`	Environment

Six findings moved from product to environment. They are not dropped; they are the evidence that this environment cannot support a release decision. But they are no longer accusations aimed at the backend team.

Four hardened into Critical defects in the Angular client, all of them only findable by driving the real UI.

One reservation stays open. Annexe H reports TC-11 and TC-12 failing on **preprod**, with an authenticated Member reading another user's data. Preprod is not json-server and I have never seen it. Nothing here explains that away, and the demo environment cannot produce evidence either way.

---

8. Housekeeping

Testing wrote data into the shared demo environment: 23 transactions and 16 transfer records. I removed all of it. The environment is back to its original 15 transactions summing to -$389.66, with an empty transfers collection, and the dashboard reads $48,291.00 again.

Worth flagging for whoever maintains this: `DELETE` works unauthenticated too, so any candidate can wipe the demo data. Mine was recoverable because the original records use `tx-0NN` ids and I could tell them apart from what I added. A candidate who deleted an original record could not have put it back.

---

9. What changed in the deliverables

1. Page objects rewritten against verified hooks and hash routing.
2. PO question Q4 replaced. It asked whether the unauthenticated access was a demo concession; I answered it with evidence instead of spending a question. The new one asks which environment QA is supposed to validate against.
3. Q2 reframed. The balance is derived but by an unknown formula, so the useful question is what it should represent and where it should come from.
4. The recommendation stays No-Go, on firmer ground. Not "the API is insecure", which was the fixture. Rather: four Critical defects in the client, one of them a critical journey that cannot be completed, and no environment in which any server-side rule can be checked.